



KAVACH

SECURE · ANALYZE · PROTECT

Progress Report

UCO Bank Hackathon — Team Kavach

AI-Powered DevSecOps Platform for Banking & Financial Applications

MVP LIVE ✓

What We Set Out to Build

Problem Statement — Digital Asset Protection in Banking



Banking Security Gap

Existing SCA tools treat a CVSS 9.8 in payments the same as one in an error logger — missing real business context entirely.



Compliance Burden

Developers manually translate scan results into RBI IT Framework 2021, PCI-DSS 4.0, and SWIFT CSP clauses — costing weeks per audit.



No Zero-Day Visibility

All tools wait for a CVE to be published. Banking applications need predictive risk scoring before exploits go public.

What We Built

Full-stack DevSecOps prototype — frontend, backend, AI engine, three scanners

Frontend

- React 18 + TypeScript + Vite
- Tailwind CSS v4 + Framer Motion
- Real-time scan dashboard
- AI Insights panel
- Compliance & BRS views

Backend

- FastAPI + Python 3.11
- Async orchestration layer
- Thread-safe JSON persistence
- PDF / SARIF / SBOM export
- REST API for frontend

AI & Scan

- Google Gemini AI engine
- Semgrep static analysis
- Pip-Audit dependency scan
- Pattern-based config scan
- BRS calculation engine

Key Features — Implemented



Banking Risk Score (BRS)

Proprietary metric: CVSS × Module Criticality Weight.
Payments modules weighted 3×, auth 2.5×, logging 1×.



Regulatory DNA Mapping

Auto-maps every finding to RBI IT Framework 2021, PCI-DSS v4.0, and SWIFT CSP clauses — zero manual effort.



AI-Powered Insights

Google Gemini translates technical CVEs into plain-language business impact and provides actionable fix suggestions.



Zero-Day Risk Prediction

Analyses outdated dependencies and high-risk code patterns to estimate unknown vulnerability probability before CVEs drop.



Three Scanner Pipeline

Semgrep (AST static analysis) + Pip-Audit (dependency CVEs) + Pattern-based config scanner — all in one CI/CD gate.



Professional Reporting

Generates Executive PDF, SARIF 2.1 security reports, and CycloneDX SBOM (JSON/XML) from a single scan run.

Technology Stack

Core Engine

Python 3.11 · FastAPI · asyncio · Semgrep · Pip-Audit

AI / ML Layer

Google Gemini API · scikit-learn · spaCy · BRS Engine

CVE & Advisory Intel

NVD REST API · OSV.dev API · GitHub Advisory DB · CVSS v3.1

Frontend

React 18 · TypeScript · Vite · Tailwind CSS v4 · Framer Motion · Recharts

Storage & Infra

Thread-safe JSON persistence · Redis-ready · Docker Compose · PostgreSQL-ready

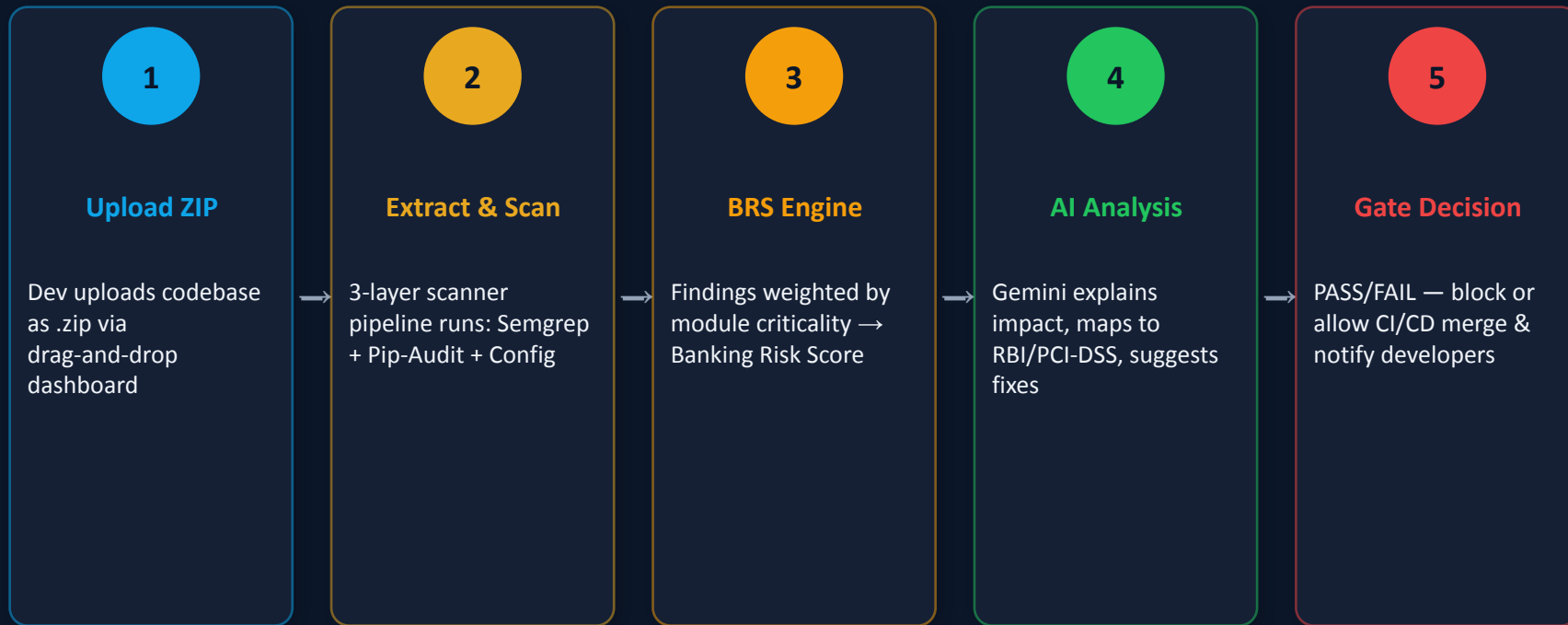
Output Formats

SARIF 2.1 · CycloneDX SBOM (JSON/XML) · PDF Audit Reports · REST API



MVP Live at: <https://strong-conkies-d5f489.netlify.app/>

How KAVACH Works — Scan Pipeline



Outputs generated: PDF Audit Report · SARIF 2.1 · CycloneDX SBOM · Jira Tickets · GitHub/GitLab Security Tab

MVP — What's Working Today

Live at: <https://strong-conkies-d5f489.netlify.app/>



Completed & Working

- Full-stack app running locally (backend + frontend)
- ZIP upload → extract → multi-scanner pipeline
- Semgrep AST scan for secrets, weak ciphers, misconfigs
- Pip-Audit CVE scan on requirements.txt / pyproject
- Pattern-based YAML/ENV/JSON config scanner
- Banking Risk Score (BRS) calculation per finding
- Google Gemini AI explanations + fix recommendations
- Regulatory mapping to RBI, PCI-DSS, SWIFT, ISO 27001
- Netlify-deployed frontend demo live



Reports & Outputs

- Executive PDF Audit Report (ReportLab, auto-generated)
- SARIF 2.1 export for GitHub/GitLab Security tab
- CycloneDX SBOM in JSON and XML
- BRS heatmap — payments vs auth vs logging modules
- Regulatory DNA view — clause-level compliance gaps
- AI-generated taint flow analysis narratives
- CI/CD Gate decision: PASS / FAIL with dev notification

How KAVACH Differs from Existing Tools

Capability	Traditional SCA Tools	KAVACH
Risk scoring	Flat CVSS — payments = logging	$BRS = CVSS \times \text{Module Criticality Weight}$
Data flow analysis	Flag where secrets exist in code	Multi-hop taint tracking — how they leak
Compliance output	Vulnerability list — manual mapping	Auto-mapped to RBI, PCI-DSS, SWIFT clauses
Zero-day prediction	Waits for CVE publication	ML model predicts risk before CVE drops
Developer experience	Separate tools, manual steps	One CI/CD gate — auto-fix PRs + rollback
AI integration	None / basic summarization	Gemini explains business impact + fix steps

Future Roadmap

Phase 2

IDE Plugin Support

Real-time alerts directly inside VS Code and IntelliJ — catch issues before code even hits the pipeline.

Phase 2

Multi-Language Expansion

Current: Python + Java. Planned: Go, Rust, JavaScript/TypeScript — full modern banking stack.

Phase 3

LLM Code Review Assistant

Conversational AI where devs ask 'why is this flagged?' and get plain-English explanation + impact + fix.

Phase 3

Runtime Threat Correlation

Connect static findings to live app telemetry — validate which vulnerabilities are actually exploited in production.

Phase 4

Threat Intelligence Feeds

MITRE ATT&CK + CISA KEV integration to prioritize findings that are actively weaponized in the wild.

Phase 4

FinTech SaaS Model

Multi-tenant SaaS packaging for smaller banks, NBFCs, and fintech startups without enterprise security budget.



Import favourites



Gmail



YouTube



Maps



WhatsApp



ALLEN Plus



Electronic library....



New Tab



Other favourites

**KAVACH**

SYS.ACTIVE

BANKING SECURITY COMMAND CENTER

Kavach

↑ SOURCE CODE INGESTION

UPLOAD LOCAL REPOSITORY ZIP ARCHIVE FOR ANALYSIS



INITIATE SECURITY ASSESSMENT

DROP A REPOSITORY ARCHIVE TO BEGIN BANKING THREAT ANALYSIS

SELECT SOURCE PAYLOAD

> PLATFORM READINESS FEED

REAL-TIME LOCAL OPERATION TELEMETRY

- > Compliance standard loaded: PCI-DSS v4.0 Requirement 6.4.3
- > Compliance standard loaded: SWIFT Customer Security Programme (CSP) CS F-v2025
- > Supply-chain audit engine loaded: CycloneDX SBOM aggregator
- > Static analysis compiler: Semgrep core rules loaded
- > Secrets detection: High-entropy regex dictionary cached
- > BRS Algorithm initialized: Risk weighting indices compiled
- > Banking Risk Matrix: Threat likelihood model online
- > Threat correlation network: Nodes mapped to payments, auth, data, compliance, infra, secrets

PLATFORM STATUS: ACTIVE

SECURED



PRE-MADE SANDBOX PAYLOADS

READY-MADE TEMPLATES TARGETING DIFFERENT SECURITY RISK CLASSES. SELECT TO TRIGGER A SANDBOX ASSESSMENT INSTANTLY.



LOW RISK SANDBOX

Standard production API deployment with fully updated libraries, parameterized database

< 15 BRS



MEDIUM RISK SANDBOX

Semi-secure banking interface utilizing legacy hash functions, non-cryptographic random

15 - 45 BRS



HIGH RISK SANDBOX

Critical vulnerability environment containing hardcoded AWS secret keys, database

> 45 BRS